# COS40003 Concurrent Programming: Learning Journal I

**Course**: COS40003 Concurrent Programming
**Assessment**: Learning Summary Report / Learning Journal I
**Student Name**: Show Wai Yan
**Date**: October 2026

### Overview

This learning journal documents my synthesis of concurrent programming concepts, operating system mechanisms, and practical synchronization techniques across Weeks 2 through 8. Each chapter summarizes core lecture concepts, reflects on weekly lab exercises, and explores self-directed research beyond the slides.

**Table of Contents**


- **Week 2 (Lecture 1): Introduction to Concurrent Programming**
- **Week 3 (Lecture 2): Processes & CPU Virtualization**
- **Week 4 (Lecture 3): CPU Scheduling**
- **Week 5 (Lecture 4): Threads I**
- **Week 6 (Lecture 5): Threads II**
- **Week 7 (Lecture 6): Lock I**
- **Week 8 (Lecture 7): Lock II**


## Week 2 (Lecture 1): Introduction to Concurrent Programming

### 1. Sequential vs. Concurrent Programming

Before this unit, most software I wrote was sequential. In **sequential programming**, instructions execute strictly in written order with no temporal overlap, making execution **deterministic**—the same input always produces the exact same execution sequence.

In contrast, **concurrent programming** breaks a problem into smaller subtasks whose execution periods overlap in time. Because the operating system scheduler interleaves these subtasks dynamically, concurrent execution is inherently **non-deterministic**. Without proper synchronization, execution order cannot be guaranteed, which can introduce bugs like race conditions and deadlocks.

### 2. Significance & Real-World Applications

Concurrency is essential in modern systems for three primary reasons:

1. **Masking I/O Latency**: In sequential execution, the CPU idles during disk reads or network transfers. Concurrency allows the CPU to switch to another ready task, maximizing utilization.
2. **UI Responsiveness**: Running heavy calculations in the background prevents user interfaces from freezing.
3. **Multi-Core Exploitation**: Modern CPUs scale through multiple cores; concurrent software is required to utilize all available execution units.

**Application Examples**:

- *Web Servers (e.g., Apache, Nginx)*: Handle thousands of simultaneous client HTTP requests concurrently instead of forcing requests into a slow sequential queue.
- *GUI Download Managers (e.g., Canvas slide downloads)*: Fetch large files in background threads while keeping navigation smooth and responsive.
- *Media Players*: Concurrently stream data from disk, decode video/audio frames, and render output while handling user play/pause inputs.

### 3. Concurrency vs. Parallelism (Observer vs. System View)

The lecture establishes a clear distinction based on perspective:

- **Concurrency (Observer's View)**: The application appears to process multiple tasks simultaneously over overlapping periods. This can run on a **single-core CPU** via rapid **time-sharing** (interleaving). Concurrency is about *program structure* (dealing with lots of things at once).
- **Parallelism (System's View)**: Multiple tasks physically execute at the exact same physical clock cycle. This strictly requires **multi-core CPUs or GPUs**. Parallelism is about *hardware execution* (doing lots of things at once).

*The Warehouse Analogy*: Moving boxes with 1 worker and 1 trolley is sequential. 1 worker alternating between 2 trolleys is single-core concurrency (two jobs in flight, one moving at a time). 2 workers pushing 2 trolleys simultaneously is parallelism. Scaling to 20 workers increases throughput, but causes door bottlenecks without synchronization rules.

### 4. Computing Paradigms

In Lecture 1 and Lab 1, we surveyed seven key computing paradigms:

- **Multiprogramming**: Simulates concurrency on a single processor via rapid time-sharing to avoid CPU idle time during I/O.
- **Multiprocessing**: True physical parallelism across multiple CPU cores sharing common physical memory (RAM).
- **Distributed Computing**: Autonomous computers with private memory communicating strictly via network message passing (e.g., sockets, MPI).
- **Cluster Computing**: Co-located, homogeneous machines connected via high-speed LAN acting as a single high-performance system.
- **Grid Computing**: Geographically dispersed, heterogeneous computers loosely sharing capacity over the Internet (largely superseded by cloud).
- **Cloud Computing**: On-demand virtualized computing delivered over the web across IaaS (virtual infrastructure), PaaS (deployment platforms), and SaaS (ready software).
- **Fog / Edge Computing**: Decentralized processing placed close to IoT sensors and edge devices to cut latency and bandwidth.

### 5. Lab 1 Practical Experience: Build Cycles & Thread Sleep

In Lab 1, I put these principles into practice using C and Java. For this exercise, I implemented `CountdownTimer.java`:

```java
public class CountdownTimer {
  public static void main(String[] args) throws InterruptedException {
    for (int i = 5; i > 0; i--) {
      System.out.println("Time remaining: " + i + " seconds");
      Thread.sleep(1000);
    }
    System.out.println("Countdown complete!");
  }
}
```

From this exercise, I noted key practical behaviors:

- **Build Models**: While C compiles directly to native binaries (`gcc -pthread -o main main.c`), Java uses a two-stage process: `javac CountdownTimer.java` compiles to bytecode (`.class`), executed by the JVM (`java CountdownTimer`).
- **OS Sleep Mechanics**: `Thread.sleep(1000)` signals the OS scheduler to transition the main thread from running to a timed-waiting state, relinquishing the CPU. Watching the terminal pause 1 second between prints visually demonstrated scheduler-enforced time delays.

- **Cooperative Interruption**: Because Java avoids forcibly killing threads, `Thread.sleep()` throws an `InterruptedException` if interrupted. As a checked exception, Java requires handling it; my code declared `throws InterruptedException` on `main`, cleanly passing the signal up to the JVM runtime.

### 6. Beyond the Slides: Extra Paradigms Researched

Answering the lab prompt to find related computing paradigms beyond the slides, I researched:

1. **Serverless Computing (FaaS, e.g., AWS Lambda)**: Instead of renting 24/7 cloud servers, developers upload discrete functions executed only when triggered by events. The provider automatically scales concurrency from zero to thousands of instances, billing per millisecond of compute time.
2. **GPU Computing (GPGPU)**: While CPUs use few powerful cores optimized for complex sequential logic, GPUs contain thousands of smaller cores for SIMD (Single Instruction, Multiple Data) parallelism. This makes GPUs the dominant architecture for massive parallel matrix computations in machine learning and rendering.

### 7. Key Takeaways

Concurrency is an architectural pattern for structuring independent tasks, while parallelism is physical hardware execution. Single-core time-sharing delivers responsiveness and hides I/O latency, but true execution speedup requires parallel multi-core hardware.

## Week 3 (Lecture 2): Processes & CPU Virtualization

### 1. What is a Process & Address Space

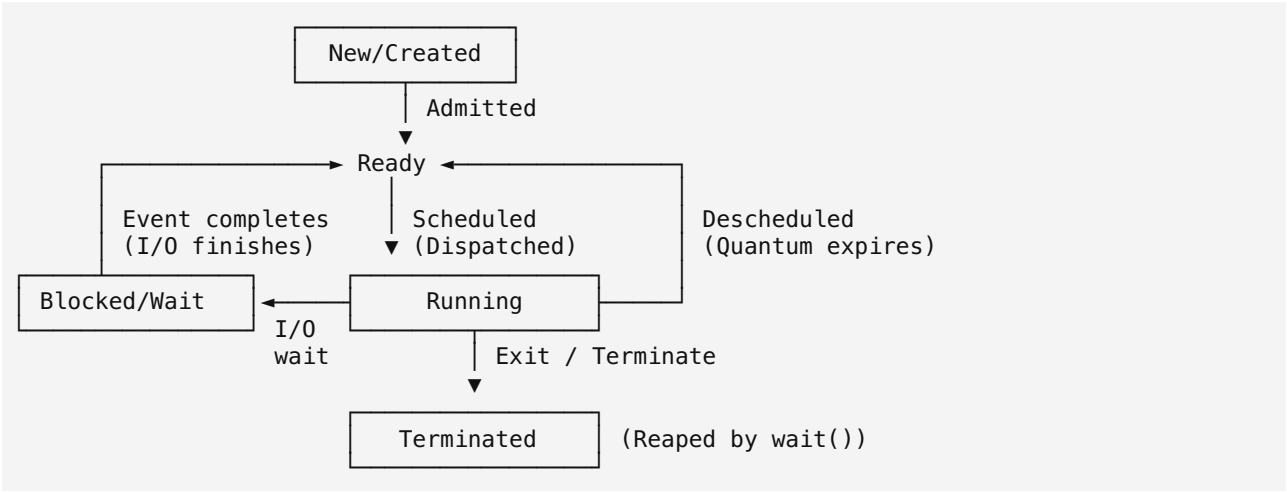
In Week 3, we examined how the operating system virtualizes the CPU to create the illusion of multiple simultaneously running programs.

- **Program vs. Process**: A program is a lifeless sequence of compiled instructions stored on disk. A **process** is an active, executing instance of that program loaded into memory.
- **Process Address Space**: When a program runs, the OS allocates an isolated address space containing four key segments:
  1. *Text / Code*: Read-only machine instructions executed by the CPU.
  2. *Data*: Global and static variables initialized before runtime.
  3. *Heap*: Dynamically allocated memory (`malloc()` in C, `new` in Java) that grows upward as memory is requested.
  4. *Stack*: Local variables, parameters, and return addresses that grow downward with each function call.
- **Process Control Block (PCB)**: The OS kernel tracks each process via its PCB, storing the Process ID (PID), current state, program counter (PC), CPU registers (SP, general registers), open file descriptors, and scheduling priority.

### 2. Process Lifecycle: 5 States & Transitions

A process progresses through five states during its execution:

1. **New / Created**: The OS allocates the PCB and initializes memory structures.
2. **Ready**: The process is loaded in RAM, waiting in the ready queue for CPU time.
3. **Running**: Instructions are actively executing on CPU hardware.
4. **Blocked / Waiting**: The process is suspended waiting for an external event (e.g., disk I/O, child exit).
5. **Terminated / Final (Exit)**: The process has finished execution but remains in a "zombie" state until its parent reaps its exit code via `wait()`.




### 3. Context Switch Mechanism vs. Scheduling Policy

A crucial distinction from Lecture 2 was separating low-level mechanisms from high-level policies:

- **Context Switch Mechanism**: The low-level hardware/kernel routine that switches the CPU from one process to another. When an interrupt occurs, the OS halts Process 1, saves its registers and PC into its PCB, restores Process 2's saved registers from its PCB into the CPU, and switches memory pointers. This is **pure overhead** because the CPU performs bookkeeping rather than user code.
- **Scheduling Policy**: The high-level algorithm deciding *which* ready process runs next, balancing interactive response time against overall throughput.

### 4. Lab 2 Practical Observations: Process APIs in C

In Lab 2, I explored UNIX process APIs by writing and testing several small C programs:

- **`fork()` (`forkExample1.c`)**: Duplicates the calling process. It returns `0` in the child, returns the child's PID in the parent, and `<0` on error. In `forkExample2.c`, the parent spawned two children printing numbers and letters concurrently, showing interleaved execution.
- **`wait()` (`forkExample3.c`)**: `wait(&childReturnValue)` pauses the parent until the child terminates. Because UNIX packs exit details, the returned integer must be bit-shifted (`childReturnValue >> 8`) to retrieve the child's actual exit status.
- **`exec()` (`execExample.c`)**: Calling `execl("./helloworld", "helloworld", NULL)` performs a "brain transplant"—it clears the child's address space and overwrites it with the `helloworld` binary. Code placed after a successful `execl()` never runs.
- **FAQ Output Buffering**: In `Lecture 2 - FAQ`, the lecturer noted that calling `printf` without `\n` or `fflush(stdout)` leaves unflushed text in the stdout buffer. When `fork()` clones memory, that buffer is duplicated, producing duplicate terminal output.

### 5. Beyond the Slides: Linux Namespaces & Cgroups

To expand beyond course materials, I researched how modern containers (like Docker) build on process isolation:

- **Linux Namespaces**: While normal processes share a global view of PIDs and mounts, namespaces partition resources so a process group sees an isolated workspace. A process inside a PID namespace can have PID 1 (believing it is the system root process) while isolated from the host OS.
- **Control Groups (cgroups)**: While namespaces isolate *what* a process can see, cgroups limit *how much* hardware resources (CPU, RAM, I/O) it can consume, preventing runaway processes from crashing the system. Together, namespaces and cgroups enable lightweight container virtualization without the overhead of booting separate virtual machine operating systems.

### 6. Key Takeaways from Week 3

A process encapsulates a running program with an isolated address space and PCB state. The UNIX `fork()` and `exec()` split cleanly separates process creation from program loading, while `wait()` ensures parents properly synchronize and reap zombie child processes.

## Week 4 (Lecture 3): CPU Scheduling

### 1. Core Tasks of the Scheduler & Metrics

In Week 4, we studied how the operating system shares the CPU among ready processes:

- **Role of the Scheduler**: Decides *which* ready process runs, *when*, and for *how long*. Decisions occur when processes are created, block on I/O, finish I/O, terminate, or are interrupted by hardware timers.
- **Preemptive vs. Non-Preemptive**: Non-preemptive scheduling lets a process run until it voluntarily yields or terminates. **Preemptive scheduling** allows the OS to interrupt a running process at any time to reallocate the CPU.
- **Evaluation Metrics**:
  - *Turnaround Time*: Total time from arrival to completion ($T_{turnaround} = T_{completion} - T_{arrival}$). Measures batch throughput efficiency.
  - *Response Time*: Time from arrival to first execution ($T_{response} = T_{first_run} - T_{arrival}$). Critical for interactive responsiveness.
  - *Fairness vs. Performance*: Optimizing turnaround often conflicts with equitable CPU distribution and quick response times.

### 2. Classic Scheduling Algorithms & Trade-offs

The lecture introduced foundational policies by relaxing initial workload assumptions:

- **FIFO / FCFS**: Non-preemptive and simple. Suffers from the **Convoy Effect**: when a long CPU-bound job runs first, short jobs get trapped behind it, inflating average turnaround time.
- **SJF (Shortest Job First)**: Non-preemptive. Runs the shortest job first, achieving optimal average turnaround time under simultaneous arrivals. However, if a long job arrives first, newly arrived short jobs must still wait.
- **STCF (Shortest Time-to-Completion First)**: Preemptive SJF. Preempts the running job if a newcomer has a shorter remaining runtime. It optimizes turnaround time but can starve long tasks and yields poor response times.
- **Round Robin (RR)**: Preemptive and time-sliced. Processes run cyclically for a fixed **time quantum**.
  - *Short Quantum*: Great response time for interactive jobs, but incurs high context-switching overhead.
  - *Long Quantum*: Amortizes context-switch costs, but degrades response time (approaches FIFO if too large).

### 3. Multi-Level Feedback Queue (MLFQ)

MLFQ is an adaptive policy used in modern operating systems (BSD, Solaris, Windows) that learns process behavior dynamically without prior runtime knowledge.

- **The 5 Rules**:
  1. Priority(A) > Priority(B) $\rightarrow$ A runs.
  2. Priority(A) = Priority(B) $\rightarrow$ A and B run in Round Robin.
  3. New jobs enter at the topmost (highest priority) queue.

4. **Anti-Gaming**: Once a job consumes its allotted time budget at a given level (even across voluntary I/O yields), it is demoted one queue.
5. **Priority Boost**: After time period SS, move all jobs back to the topmost queue.

- **Why MLFQ Works**: Interactive I/O jobs issue short bursts, staying at top queues with short time slices for fast response times. Long batch jobs consume full time slices and demote to lower queues with longer slices, maximizing CPU throughput.
- **Fixing Flaws**: Rule 5 (Priority Boost) prevents starvation of lower-queue batch jobs. Rule 4's cumulative accounting prevents malicious processes from gaming the scheduler by yielding the CPU at 99% of their quantum to retain top priority.

### 4. Lab 3 Tutorial Insights: Starvation & Stride Scheduling Case Study

In the Lab 3 tutorial, we analyzed scheduling edge cases and proportional-share fairness:

- **Starvation Analysis**:
  - *Immune*: **FCFS** (jobs always reach the front) and **Round Robin** (jobs receive regular slices).
  - *Vulnerable*: **SJF**, **STCF**, and **Fixed Priority** (continuous short/high-priority jobs starve long tasks). **MLFQ** starves batch jobs unless Rule 5 (Priority Boost) is enforced.
- **Case Study: Lottery vs. Stride Scheduling**:
  - *Lottery Scheduling*: Randomized ticket drawing. Stateless and simple to add new jobs, but random variance produces inaccurate proportions over short time scales.
  - *Stride Scheduling*: Deterministic fair-share. Stride=Large ConstantTickets\text{Stride} = \frac{\text{Large Constant}}{\text{Tickets}} (e.g., 10,000/tickets10,000 / \text{tickets}). Each run increments a process's `pass` value by its stride; the scheduler always picks the lowest `pass`.
  - *Trade-off*: Stride delivers exact proportions over short intervals, but requires global state. If a new job enters with pass=0\text{pass} = 0, it unfairly monopolizes the CPU until its pass catches up.

### 5. Beyond the Slides: Linux Completely Fair Scheduler (CFS)

To explore modern production schedulers, I investigated the **Linux Completely Fair Scheduler (CFS)**:

- **Virtual Runtime (`vruntime`)**: CFS tracks each task's CPU time scaled by its `nice` priority in a Red-Black Tree (O(logN)O(\log N) selection), continuously picking the task with the lowest `vruntime`.
- **Solving the New Process Monopolization**: To resolve the exact flaw we studied in Stride scheduling (where a newcomer with pass 0 monopolizes the CPU), CFS sets a new task's initial `vruntime` to `min_vruntime` (the minimum virtual runtime of all active tasks in the tree), ensuring fair integration.

### 6. Key Takeaways from Week 4

CPU scheduling balances turnaround time (batch efficiency) against response time (interactivity). Real-world schedulers like MLFQ and Linux CFS demonstrate that systems can adaptively prioritize interactive tasks while guaranteeing starvation-free fairness for background compute workloads.

## Week 5 (Lecture 4): Threads I – Execution Contexts & Concurrency Models

### 1. Thread Abstraction vs. Processes

In Week 5, we shifted focus from operating system multitasking across separate processes to concurrency *within* an individual process:

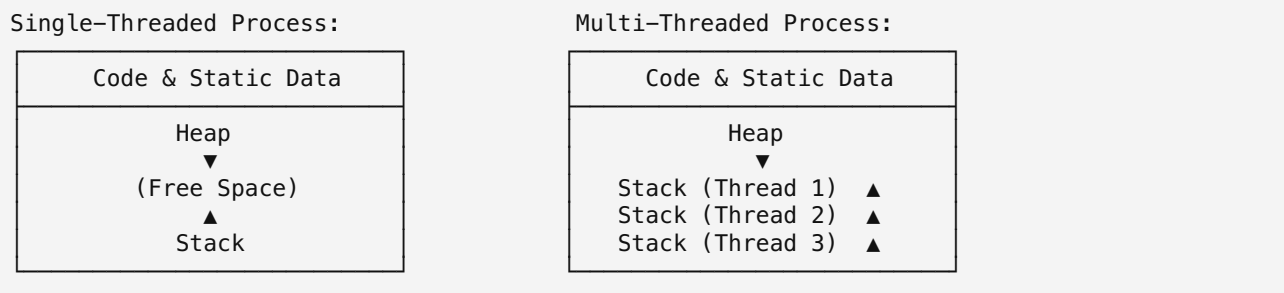
- **Process vs. Thread**: A process is the basic unit of **resource allocation**, owning an isolated address space, file descriptors, and security credentials. A **thread** is the basic unit of **CPU scheduling**, representing a single independent control-flow stream within that process.

- **Why Threads Exist**: Splitting an application into multiple processes via `fork()` incurs significant overhead due to memory duplication and slow Inter-Process Communication (IPC). Threads solve this by executing concurrently inside the **same address space**, providing low-overhead task switching and keeping user interfaces responsive during background I/O operations.

### 2. Address Space: Shared vs. Private State

Unlike a single-threaded process that contains a single stack growing toward a single heap, a multi-threaded process alters the memory layout:

- **Shared Across Threads**: All threads belonging to the same process share the **Text Segment (program code)**, **Data Segment (global/static variables)**, **Heap (dynamically allocated memory)**, open file descriptors, network sockets, and Process ID (PID).
- **Private Per-Thread State**: Each thread maintains its own independent execution context:
  - Program Counter (PC) tracking instruction execution.
  - Stack Pointer (SP) and general-purpose CPU registers.
  - Dedicated **Private Stack** allocated within the process memory space for local variables, parameters, and function frames.




### 3. User-Level vs. Kernel-Level Threads

The lecture distinguished where threads are managed. **User-level threads** are created and scheduled by a library in user space: switching is cheap (no system call), but the kernel sees only one schedulable entity, so one blocking I/O call stalls every thread of the process and threads cannot run on several cores. **Kernel-level threads** are known to the OS scheduler: each can block independently and run on a different core, at the cost of a system call for creation and switching. Modern systems (Linux `pthread`, Java platform threads) use a 1:1 model mapping each thread to a kernel thread, which is why Java threads can exploit multicore hardware.

### 4. Thread Design Patterns & Programming APIs

The lecture highlighted standard concurrent architectures and APIs:

- **Design Patterns**:
  - *Manager / Worker*: A manager thread handles external events or I/O and delegates tasks to worker threads.
  - *Pipeline*: Sequential stages organized like an assembly line, where each thread processes data and passes it to the next stage.
- **POSIX C APIs (`pthread`)**: `pthread_create()` creates a new thread running a specified start function, `pthread_exit()` terminates the thread, `pthread_join()` blocks the caller until the target thread finishes, and `pthread_yield()` voluntarily gives up CPU execution.
- **Java Thread Creation**:
  - *Extending `Thread` vs. Implementing `Runnable`*: Implementing `Runnable` is preferred because Java only allows single class inheritance. Passing a `Runnable` cleanly separates the task logic from the thread execution mechanism.

- *Crucial Rule (`start()` vs. `run()`)*: In `BasicThread.java`, I verified that calling `.start()` instructs the JVM to spawn a new OS-level thread. Calling `.run()` directly simply executes the code synchronously within the caller's thread without any concurrency.

### 5. Lab 4 Practical Observations: Variable Scopes & Thread Join

In Lab 4, I analyzed variable visibility and thread coordination in Java:

- **Memory Scopes and Race Conditions (`ConcurrencyProblem.java`)**:
  - `localData`: Stored on each thread's private stack →\rightarrow independent across threads and perfectly deterministic (consistently reaching 1,000,000).
  - `instanceData`: Stored on the heap →\rightarrow isolated when threads run separate instances, but shared if threads share an object.
  - `classData` (`static`): Stored in shared class memory →\rightarrow accessed concurrently by all threads without synchronization, resulting in race conditions and severely lost updates.
- **Thread Coordination via `join()` (`JoinThread.java`)**: In `JoinThread.java`, a worker thread calculates the average of an array. By calling `t1.join()`, the main thread blocks until `t1` completes, guaranteeing the result is ready before printing. Removing `join()` causes the main thread to read an uninitialized average, demonstrating why synchronization is essential.

### 6. Beyond the Slides: Hardware Simultaneous Multithreading (SMT / Hyper-Threading)

To explore how physical hardware supports multithreading, I researched **Simultaneous Multithreading (SMT)**:

- **Concept**: In an SMT processor core (such as Intel Hyper-Threading), a single physical CPU core duplicates its architectural state (Program Counters, registers, and execution contexts) so the operating system sees two logical processors.
- **Shared Silicon Pipelines**: However, the underlying physical execution units (ALUs, floating-point units, and L1/L2 caches) are physically shared. When Thread A stalls waiting for data from main memory, Thread B immediately utilizes the execution pipeline rather than leaving the core idle.
- **Performance Implication**: SMT typically provides a 20% to 30% throughput increase rather than doubling speed, because sibling hardware threads contend for the same physical execution units and cache lines.

### 7. Key Takeaways from Week 5

Threads enable lightweight concurrency by sharing memory within a process, avoiding the heavy context-switching overhead of processes. However, as demonstrated by the corruption of `classData` in Lab 4, shared mutable memory creates dangerous race conditions, proving that concurrency without synchronization leads to non-deterministic failure.

## Week 6 (Lecture 5): Threads II – Thread Pools, the Executor Framework & Virtual Threads

### 1. Limitations of the Thread-per-Task Model

In Week 5, we created threads on demand using `new Thread()`. Week 6 highlighted why this naive "thread-per-task" model fails under production server loads:

- **Creation & Stack Overhead**: Creating native OS threads incurs high CPU setup latency and heavy memory consumption (~1 MB stack per thread in 64-bit JVMs). Spawning thousands of short-lived threads wastes CPU cycles on stack allocation rather than task computation.
- **Resource Exhaustion & Thrashing**: Under sudden traffic spikes, unbounded thread creation quickly causes memory exhaustion (`OutOfMemoryError`). Furthermore, having thousands of active threads forces the OS into relentless context switching and cache thrashing, degrading throughput.

### 2. Thread Pool Architecture & Sizing Dynamics

Thread pools solve this by decoupling task submission from execution via a producer-consumer model:

- **Mechanism**: A pool pre-allocates reusable worker threads and a task queue (`BlockingQueue`). Workers wait for tasks, execute them, and return to the pool rather than terminating.
- **Sizing Trade-offs**: Oversized pools cause memory exhaustion and context-switching churn; undersized pools starve clients with high queuing delays. Compute-bound tasks target Nthreads=NCPU+1N_{\text{threads}} = N_{\text{CPU}} + 1, while I/O-bound tasks scale with wait-to-compute ratios.
- **Queue Bounds**: The slides emphasized bounding work queues to prevent unbounded memory growth and using explicit rejection policies when overloaded.

### 3. The Java Executor Framework & Result-Bearing Tasks

Java standardizes thread pool management via `java.util.concurrent`:

- **`ExecutorService` & `ThreadPoolExecutor`**: Instead of raw threads, applications configure `ThreadPoolExecutor` (specifying `corePoolSize`, `maximumPoolSize`, `keepAliveTime`, `workQueue`, and rejection handlers) or use factory shortcuts like `Executors.newFixedThreadPool(n)`.
- **Result-Bearing Tasks (`Callable<V>` & `Future<V>`)**: Unlike `Runnable.run()` which returns `void`, `Callable<V>` defines `V call() throws Exception` to return values. Submitting a `Callable` yields a `Future<V>`, enabling callers to asynchronously poll status or call `future.get()` to retrieve results safely.
- **Lifecycle Discipline**: Pools must be shut down with `.shutdown()` and `.awaitTermination()` to prevent task loss and resource leaks.

### 4. Thread Safety

A class or method is **thread-safe** if it behaves correctly when accessed from multiple threads at once, with no extra coordination by callers. Pools make this important: worker threads are reused, so any shared state they touch (counters, caches, result lists) is accessed concurrently. Thread safety is achieved by avoiding shared mutable state (each task uses local variables on its private stack, or immutable data), or by protecting shared state with locks or atomic variables (Weeks 7 and 8). The lab design reflects this: tasks returned results through `Future` instead of writing into a shared variable.

### 5. Lab 5 Practical Observations: From Thread Pools to Virtual Threads

In Lab 5, I explored thread pool tuning and compared execution models across real workloads:

- **Configuring Pools (`ThreadPoolExecutorExample.java`)**: In Lab 5-2, I configured a `ThreadPoolExecutor` (core size 2, maximum 3, 60s keep-alive) using a custom `MyThreadFactory` naming threads `"Worker_Thread"`. Calculating median sets (`Median.java`) showed worker threads being cleanly recycled across tasks.
- **Asynchronous Results (`CallableExample.java`)**: In Lab 5-3, I submitted median tasks as `Callable<Double>` to an `ExecutorService` with 3 threads. Using `Future<Double>` and `future.get()` demonstrated how futures eliminate ad-hoc shared memory synchronization.
- **Benchmarking 4 Paradigms (`FactorFinder` in Lab 5-4)**: When factoring 1,000 large integers (5M to 8M), I evaluated four approaches:
  1. *Sequential*: Single-threaded execution suffered from long serial latency.
  2. *Unbounded Threads (`FactorFinder_Threaded.java`)*: Spawning 1,000 OS threads incurred heavy creation and teardown overhead.
  3. *Fixed Thread Pool (`FactorFinder_ThreadPool.java`)*: Bounding concurrency to available CPU cores eliminated churn and maximized throughput.

4. *Virtual Threads (`FactorFinder_VirtualThread.java`)*: Spawning 1,000 Java 21 virtual threads via `Thread.startVirtualThread()` delivered near-instant creation with negligible memory overhead, coordinating cleanly with `t.join()`.

### 6. Beyond the Slides: Virtual Thread Continuations & Carrier Thread Unmounting

To understand Java 21 Virtual Threads introduced in the slides, I researched **Project Loom internals**:

- **Continuations & Carrier Threads**: Unlike an OS platform thread whose 1 MB stack is pinned in OS memory, a virtual thread is a user-space **Continuation**. When computing, the JVM mounts the continuation onto a standard OS platform thread called a **Carrier Thread** (from a shared `ForkJoinPool`).
- **Unmounting on Blocking I/O**: When a virtual thread blocks (network I/O, sleep), the JVM intercepts the call and **unmounts** the continuation, copying its execution frames to heap memory. The underlying carrier thread is instantly released to execute other virtual threads. Once the I/O finishes, the continuation is rescheduled onto an idle carrier thread.
- **Architectural Impact**: This cooperative scheduling allows millions of concurrent tasks to run using simple synchronous code without OS thread stack limits or thread-per-client scalability bottlenecks.

### 7. Key Takeaways from Week 6

Thread pools bound resource consumption and prevent server crashes under load. Furthermore, Java's shift toward virtual threads demonstrates how user-space continuations reconcile the simplicity of synchronous programming with the massive scalability required by modern concurrent systems.

## Week 7 (Lecture 6): Lock I – Mutual Exclusion, Critical Sections & Java Locks

### 1. The Critical Section Problem & Concurrency Terminology

In Week 7, we addressed protecting shared mutable data from concurrent corruption. Even simple statements like `Y = Y + 1` or `i++` are not atomic; they decompose into three machine instructions:

1. **Load**: Copy memory into a CPU register (`mov Addr1 R1`).
2. **Modify**: Increment the register (`add 1 R1`).
3. **Store**: Write the register back to memory (`mov R1 Addr1`).

If context switches interleave these steps across threads—e.g., Thread 1 computing `Y = Y + 1` while Thread 2 computes `Y = Y * 2`—the final result becomes non-deterministic (yielding 11 or 12). The lecture formalized core terms:

- **Critical Section**: Code accessing shared resources that must not be concurrently executed.
- **Race Condition**: Multiple threads entering a critical section simultaneously, producing non-deterministic outcomes.
- **Mutual Exclusion**: Guaranteeing only one thread executes in a critical section at any moment.
- **Atomicity**: Operations that complete indivisibly without interruption.

### 2. Low-Level Mutual Exclusion in C (POSIX Mutex)

The lecture illustrated mutual exclusion using a phone booth analogy: only one person enters while others queue outside. In C (`pthread`), this is enforced via `pthread_mutex_t`:

```
pthread_mutex_t myLock = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_lock(&myLock);   // Blocks if held
// Critical Section
pthread_mutex_unlock(&myLock); // Releases lock
```

Calling `pthread_mutex_lock()` on a held lock blocks the thread until release. The slides cautioned that omitting `pthread_mutex_unlock()` permanently deadlocks waiting threads.

### 3. Java Intrinsic Locks (`synchronized` Keyword)

Java integrates mutual exclusion directly through intrinsic locks (monitors):

- **Object Monitor**: Every Java object has an implicit intrinsic lock that is initially unowned.
- **Methods vs. Blocks**: `public synchronized void update()` locks the entire object instance (`this`), whereas `synchronized(aLock) { ... }` restricts locking strictly to critical lines.
- **Exception Safety**: `synchronized` automatically acquires and releases monitors on entry and exit, guaranteeing release even during unhandled runtime exceptions.

### 4. Java Explicit Locks: The `Lock` Interface & `ReentrantLock`

To overcome monitor limitations, Java provides explicit locks via `ReentrantLock`:

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    taskCount.incCount(); // Critical section
} finally {
    lock.unlock(); // Mandatory cleanup
}
```

Unlike `synchronized`, `ReentrantLock` requires manual release. Omitting `lock.unlock()` inside `finally` causes permanent lock leaks that block all future access.

### 5. Direct Comparison: `synchronized` vs. `ReentrantLock`

The lecture highlighted five operational differences:

1. **Flexibility**: `synchronized` must remain within one method; `ReentrantLock` allows decoupled `lock()` and `unlock()` calls across methods.
2. **Non-Blocking Polling**: `tryLock()` returns `false` if held, allowing threads to avoid blocking and perform alternate tasks.
3. **Timed Waiting**: `tryLock(timeout, unit)` enables bounded waits, preventing permanent deadlocks.
4. **Interruptibility**: Threads waiting in `lockInterruptibly()` can be aborted via `interrupt()`, whereas `synchronized` waits cannot.
5. **Fairness**: `new ReentrantLock(true)` grants FIFO acquisition order, whereas `synchronized` selects threads arbitrarily.

### 6. Lab 6 Practical Observations: Counter Races & Banking Simulation

In Lab 6, I applied these mechanisms to eliminate concurrency bugs:

- **Preventing Lost Updates (`MainWithLock.java`)**: In Lab 6-1, 1,000 threads incremented `TaskCounter.java`. In `MainNoLock.java`, race conditions during the load-modify-store sequence dropped the final count well below 1,000. Guarding increments with `ReentrantLock` in `try-finally` ensured exact correctness (1,000).
- **Protecting Bank Balances (`CustomerAccount.java`)**: In the banking exercise, concurrent transactions corrupted account balances. I verified that both `synchronized` methods and explicit `ReentrantLock` blocks maintain balance integrity, with `ReentrantLock` providing explicit scoping and timed polling capabilities.

### 7. Beyond the Slides: HotSpot Object Header & Monitor Inflation

To examine how Java executes intrinsic locks, I researched **HotSpot Monitor Inflation**:

- **Mark Word State**: In HotSpot, every object's 2-word header contains a Mark Word storing lock bits: `01` (unlocked), `00` (lightweight lock), and `10` (heavyweight monitor).

- **Tiered Inflation**: When an uncontended thread enters `synchronized`, the JVM uses a fast atomic CAS to store a stack pointer in the Mark Word without OS interaction. When contention occurs, the JVM inflates the lock into a heavyweight `ObjectMonitor`, allocating OS mutex primitives to park threads. This tiered design grants uncontended locks near-zero overhead while ensuring robust mutual exclusion under contention.

### 8. Key Takeaways from Week 7

Concurrent writes corrupt shared data due to multi-step load-modify-store sequences. While C uses `pthread_mutex_t` and Java offers convenient `synchronized` monitors, `ReentrantLock` provides essential control for production systems through timed, non-blocking, and interruptible locking.

## Week 8 (Lecture 7): Lock II – Building Locks from Atomic Hardware & OS Support

### 1. From Using Locks to Building Them

Week 7 taught me how to *use* `pthread_mutex_t`, `synchronized` and `ReentrantLock`. Week 8 asked how a lock is actually *built*. The lecture judged any lock by three criteria: **mutual exclusion** (one thread in the critical section), **fairness** (no contender starves), and **performance** (overhead of acquiring, contending and releasing). It concluded that a good lock needs both **hardware atomic instructions** and **OS scheduling support**.

### 2. Two Failed Attempts

- **Disabling interrupts** (`lock()` = `DisableInterrupts()`): works on a single core, but it demands excessive trust (a user program may never re-enable interrupts and monopolize the CPU), fails on multiprocessors because the interrupt flag is per-core, and can lose important interrupts such as disk completions. It is acceptable only inside the kernel.
- **A plain `flag` variable** (`while (flag == 1); flag = 1;`): *testing* and *setting* the flag are two separate steps. If Thread 1 sees `flag == 0` and is preempted before setting it, Thread 2 also sees `0`, sets it, and enters; Thread 1 then resumes and enters too. **Mutual exclusion is violated**, and the spin loop also wastes CPU.

The lesson was that software alone cannot close this gap; the test-and-set must be **one indivisible hardware operation**.

### 3. Atomic Instructions and Spin Locks

| Instruction | Atomic behaviour | Lock built from it | Fairness |
|---|---|---|---|
| **Test-And-Set (TAS)** | Return old value, store new value | `while (TestAndSet(&flag,1)==1);` | No (starvation possible) |
| **Compare-And-Swap (CAS)** | Store new only if value equals `expected`; return actual | `while (CompareAndSwap(&flag,0,1)==1);` | No |
| **Fetch-And-Add (FAA)** | Return old value, increment by 1 | **Ticket lock** (`ticket`, `turn`) | **Yes (FIFO)** |

TAS and CAS spin locks are correct but unfair. On one CPU they are very poor, since a waiter spins for its whole time slice while the holder cannot run; on multiple cores they are reasonable when critical sections are short. The lecture also noted that CAS is strictly more powerful than TAS and underpins lock-free data structures.

The **ticket lock** fixes fairness. `lock()` takes `myturn = FetchAndAdd(&ticket)` and spins until `turn == myturn`; `unlock()` does `turn++`. I traced three threads: T1 draws 0 and enters at once, T2 draws 1 and T3 draws 2 and spin; when T1 unlocks, `turn` becomes 1 so T2 enters, then T3. This gives strict FIFO order, but waiters still spin.

### 4. OS Support: Yielding and Sleeping

Pure spinning is worst when a lock holder is preempted. Two OS-assisted fixes were presented:

- **`yield()`**: on a failed TAS the thread moves from *running* to *ready*. With 100 contenders and a preempted holder, 99 threads each yield in turn, causing about 99 context switches per release. This is correct but costly under heavy contention.
- **Sleep queues with `park()` / `unpark(threadID)`**: a waiter is placed on a queue inside the lock and *blocked*; `unlock()` wakes the next one. This avoids both spinning and yield thrashing and can be fair, so it is the model behind production locks.

### 5. Where and When to Use Each Mechanism

| Mechanism | Use when | Avoid when |
|---|---|---|
| Atomic variable (`AtomicInteger`) | A single counter or flag needs updating | Several variables must change together |
| TAS/CAS spin lock | Very short critical section, multicore, few contenders | Single core, long or heavily contended sections |
| Ticket lock | Fairness and bounded waiting matter | Waiters would spin for a long time |
| `yield()` lock | Moderate contention, quick fix over spinning | Many threads (context-switch thrashing) |
| Sleep queue / `ReentrantLock` / `synchronized` | General application code, long waits, need `tryLock` or recursion | Nanosecond-scale sections where sleeping costs more than waiting |
| Interrupt disabling | Inside the kernel only | Any user program |

`ReentrantLock` from Week 7 is itself built on this stack: a CAS fast path plus a queue of parked threads, with optional fairness (`new ReentrantLock(true)`), and its recursive ownership counter lets one thread re-acquire a lock it already holds without self-deadlock.

### 6. Lab 7 Practical Observations

- **Atomic variables (Lab 7-1)**: `Jobs` used `count++` (load-modify-store, as in Week 7), while `AtomicJobs` used `AtomicInteger.getAndIncrement()`. With two threads doing 10,000 and 5,000 jobs, the atomic version always reports 15,000; the plain version can lose updates.
- **TAS spin lock (Lab 7-2)**: `SimpleLock.acquire()` is `while (flag.getAndSet(true) == true);` on an `AtomicBoolean`, a direct Java mirror of the slide's TAS. With the lock calls commented out, one thread incrementing and another decrementing 1,000,000 times each should finish at 0, yet my run printed **57525**, a visible lost-update race.
- **CAS spin lock and cost of spinning (Lab 7-3)**: I swapped in `compareAndExchange(false, true)` and measured 16 pool threads each locking 1,000 times. The count was correct (16,000), but average spin time was about **2,199 ms per thread**, and the log showed some threads waiting far longer than others, matching the lecture's claims of wasted CPU and unfairness. I also noticed the redundant `flag.set(true)` after acquiring in my code; it is harmless but unnecessary, since CAS already set it.
- **Lab exercise: fair and efficient locks**: `FairLock` implemented a ticket lock but replaced spinning with `Thread.yield()`. `EfficientLock` used `ticket`/`turn` with `LockSupport.park()` and `unpark()`. Both produced 16,000. On reflection, `EfficientLock` is fragile: it keeps one shared `HashMap` without synchronization, never uses its queue to wake threads, and has a **lost-wakeup window** (if `unlock()` runs before the waiter calls `park()`, the waiter can sleep forever unless `unpark` was already delivered). It worked in my run but is not provably safe; a correct version needs a thread-safe wait structure.

### 7. Beyond the Slides: Linux Futex and Two-Phase Locks

The slides stop at `park`/`unpark`, so I researched how real systems apply the idea:

- **Futex (fast user-space mutex)**: On Linux, `pthread_mutex_t` keeps its state in a user-space integer. An uncontended `lock()` is just one atomic CAS with **no system call**. Only on contention does the thread call `futex(FUTEX_WAIT)` to sleep in a kernel wait queue, and `unlock()` calls `FUTEX_WAKE` if waiters exist. This is the `park`/`unpark` queue idea made cheap for the common case.
- **Two-phase lock**: spin briefly first (the holder may release soon), and only then sleep. This captures the multicore benefit of spinning for short critical sections and the single-core benefit of sleeping for long ones, which is why modern mutexes mix both.
- **Link to Week 7**: HotSpot's lightweight-to-heavyweight monitor inflation is the same pattern: CAS fast path, queue and OS blocking under contention.

### 8. Key Takeaways from Week 8

Correct locks are built on atomic hardware instructions, not on plain loads and stores. TAS/CAS give mutual exclusion, ticket locks add fairness, and OS queues (`park`/`unpark`, futex) remove wasted spinning. In practice, spin for short critical sections on multicore, and block for long or highly contended ones.

## Overall Reflection & Future Learning

Across Weeks 2 to 8 the unit built one story: concurrency improves responsiveness and throughput (Weeks 2 to 4), threads make it cheap (Weeks 5 and 6), and shared memory then demands synchronization (Weeks 7 and 8). My strongest practical lesson came from the labs, where unsynchronized counters visibly produced wrong answers and spin locks visibly wasted time. My gaps are lock-free data structures built on CAS, memory models and visibility (`volatile`, happens-before), deadlock avoidance, and message-passing models such as CSP and actors. I plan to study these next, starting with `java.util.concurrent.atomic` internals and the Java Memory Model.

## References


1. Arpaci-Dusseau, R. H. and Arpaci-Dusseau, A. C., *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books (chapters on processes, scheduling, threads and locks).
2. Goetz, B. et al., *Java Concurrency in Practice*, Addison-Wesley, 2006.
3. COS40003 Concurrent Programming, lecture slides and lab sheets, Weeks 2 to 8, Swinburne University of Technology.
4. Oracle, *Java SE 21 Documentation*: `java.util.concurrent`, `java.util.concurrent.locks`, JEP 444 (Virtual Threads).
5. Linux manual pages: `futex(2)`, `pthread_mutex_lock(3)`, `sched(7)`, `namespaces(7)`, `cgroups(7)`.